



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N.º 06

NOMBRE COMPLETO: Rosas Cañada Abraham

N.º de Cuenta: 318307866

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 06

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 12/04/2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

Ejercicio 1: Crear un dado de 8 caras y texturizarlo por medio de código.



Figura 1. Octaedro texturizado creado mediante código. Las coordenadas UV mapean cada cara de la malla al número correspondiente de la imagen dado8.jpg.

```
// Indices: 8 caras triangulares del octaedro
unsigned int oct_indices[] = {
    0,1,2, 3,4,5, 6,7,8, 9,10,11,
    12,13,14, 15,16,17, 18,19,20, 21,22,23
};

// Vertices: posicion XYZ + coords UV + normales
GLfloat oct_vertices[] = {
    // cara 0 - frente superior derecho (numero 1)
    0.0f, 1.0f, 0.0f, 0.4920f, 0.7431f, 0.57f, 0.57f, 0.57f,
    0.0f, 0.0f, 1.0f, 0.2580f, 0.4982f, 0.57f, 0.57f, 0.57f,
    1.0f, 0.0f, 0.0f, 0.7240f, 0.5030f, 0.57f, 0.57f, 0.57f,
    // ... (8 caras con sus 3 vertices y UVs mapeados a dado8.jpg)
};

Mesh* dado8 = new Mesh();
dado8->CreateMesh(oct_vertices, oct_indices, 192, 24);
meshList.push_back(dado8);

// Carga de textura del dado de 8 caras
dado8Texture = Texture("Textures/dado8.jpg");
dado8Texture.LoadTextureA();

// Render en el loop principal
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(1.5f, 4.5f, -2.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
dado8Texture.UseTexture();
meshList[5]->RenderMesh();
```

Ejercicio 2: Importar el modelo de su coche con sus 4 llantas acomodadas y tener texturizadas las 4 llantas (diferenciar caucho y rin)

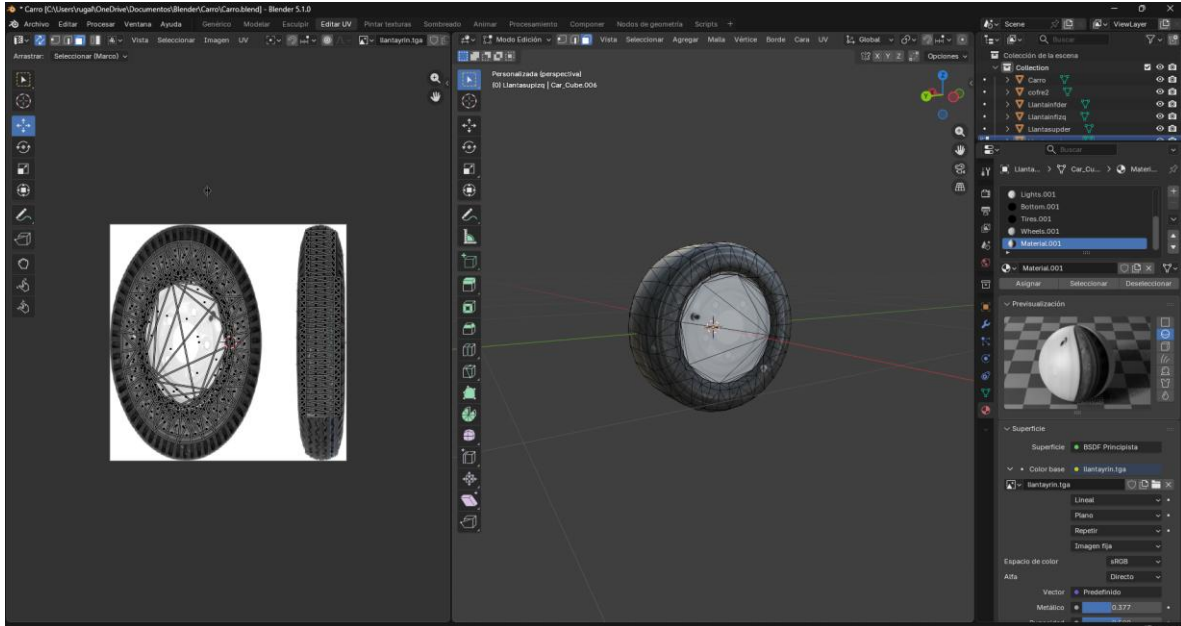


Figura 2. Modelo del coche con las cuatro llantas texturizadas importadas desde Blender. Cada llanta usa su propio archivo OBJ y comparte la transformacion base del carro a través de modelaux.

```
// Declaracion de modelos importados
Model Carrotex_M;
Model Cofretex_M;
Model Llantatexsupder_M, Llantatexsupizq_M;
Model Llantatexinfder_M, Llantatexinfizq_M;

// Carga de archivos OBJ (la textura viene referenciada en el .mtl)
Carrotex_M.LoadModel("Models/Carrotex.obj");
Llantatexsupder_M.LoadModel("Models/Llantatexsupder.obj");
Llantatexsupizq_M.LoadModel("Models/Llantatexsupizq.obj");
Llantatexinfder_M.LoadModel("Models/Llantatexinfder.obj");
Llantatexinfizq_M.LoadModel("Models/Llantatexinfizq.obj");

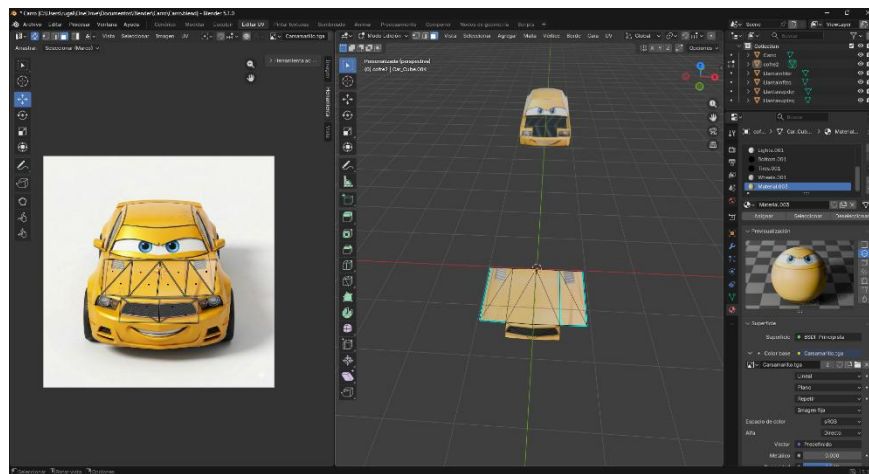
// Base comun para todo el coche (movimiento sincronizado)
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(mainWindow.getmuevex(), -0.5f, -
3.0f));
model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
modelaux = model;

// Cuerpo del carro
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 1.5f, 0.0f));
model = glm::scale(model, glm::vec3(2.5f, 2.5f, 2.5f));
```

```
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Carrotex_M.RenderModel();

// Llanta superior derecha (patron identico para las 4 llantas)
model = modelaux;
model = glm::translate(model, glm::vec3(2.5f, 0.0f, -3.85f));
model = glm::scale(model, glm::vec3(2.5f, 2.5f, 2.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Llantatexsupder_M.RenderModel(); // sup izq: X=-2.5f inf der: Z=2.50f inf
izq: X=-2.5f,Z=2.50f
```

Ejercicio 3: Texturizar la cara del personaje de la imagen tipo cars en el parabrisas (ojos) y detalles en cofre y parrilla de su propio modelo de coche





Figuras 3 y 4. Cofre y cuerpo del coche con textura Carsamarillo.tga aplicada mediante UV mapping en Blender y referenciada desde el archivo .mtl.

```
// El cofre se exporto como objeto independiente desde Blender
// El archivo .mtl referencia la textura con: map_Kd Carsamarillo.tga
Model Cofretex_M;
Cofretex_M.LoadModel("Models/Cofretex.obj");

// Fragmento del archivo Cofretex.mtl generado por Blender:
// newmtl Material.001
// map_Kd Carsamarillo.tga    <- textura UV aplicada en Blender

// Render del cofre (comparte modelaux con el carro para moverse junto)
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 1.87f, -2.65f));
model = glm::scale(model, glm::vec3(2.5f, 2.5f, 2.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Cofretex_M.RenderModel();

// El carro completo (Carrotex_M) usa la misma textura Carsamarillo.tga
// mapeada en Blender con UV Unwrap sobre la carroceria del modelo
```

2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

Durante el desarrollo de los ejercicios se presentaron los siguientes problemas:

Dado de 8 caras: El principal reto fue calcular manualmente las coordenadas UV de cada triangulo para que los numeros del dado quedaran correctamente mapeados. Se resolvió abriendo la textura dado8.jpg en GIMP y midiendo con precision los pixeles correspondientes a cada numero para obtener los valores U y V normalizados.

Coche con llantas: Al exportar el modelo desde Blender, el archivo .mtl generaba una ruta absoluta en la directiva map_Kd, lo que causaba que la textura no se cargara y el modelo apareciera completamente negro. Se resolvió editando el .mtl en el bloc de notas para dejar solo el nombre del archivo, por ejemplo map_Kd llantayrin.tga, y copiando el archivo TGA a la carpeta Textures del proyecto.

Otro problema fue que al llamar RenderModel sin haber asignado previamente la matriz model, el objeto se renderizaba con la transformacion residual del ultimo objeto dibujado. Se corrigió inicializando model desde modelaux antes de cada llamada a RenderModel.

Texturizado cofre y carro: La textura Carsamarillo.tga debia cumplir los requisitos de OpenGL: modo RGB con canal alfa, dimensiones cuadradas potencia de 2 y sin compresion. Se proceso en GIMP aplicando Imagen, Modo, RGB, agregando canal alfa con Capa, Transparencia, Agregar canal alfa, escalando a 512x512 pixeles y exportando como TGA sin compresión RLE.

3.- Conclusión:

En esta practica se comprendio de manera practica como funciona el texturizado en OpenGL: el proceso completo desde preparar una imagen en GIMP con el formato correcto (RGB + canal alfa, dimensiones potencia de 2, formato TGA), hasta mapear las coordenadas UV en los vertices del modelo y cargarlas en el pipeline de renderizado. El ejercicio del dado de 8 caras resulto especialmente util para entender que las coordenadas UV no son algo automatico, sino que requieren una correspondencia precisa entre los triangulos de la malla y las regiones de la textura. La importacion de modelos OBJ desde Blender demostro la importancia del archivo .mtl como puente entre el software de modelado y el motor grafico, y enseno a depurar errores comunes como rutas absolutas de textura que solo funcionan en la maquina del artista. Finalmente, la necesidad de usar modelaux para sincronizar las transformaciones de todos los objetos del coche refuerza el concepto de jerarquia de transformaciones visto en practicas anteriores.

Bibliografia:

- Shreiner, D., Sellers, G., Kessenich, J., & Licea-Kane, B. (2013).
- OpenGL Programming Guide: The Official Guide to Learning OpenGL (8th ed.). Addison-Wesley. / Blender Foundation. (2025).
- Blender 5.1 Reference Manual. <https://docs.blender.org> / GIMP Development Team. (2024).
GIMP User Manual. <https://docs.gimp.org>